

JOIN — соединение таблиц

Зачем нужен: аналогия из жизни

В прошлой теме студенческий билет хранит только номер группы ("IT-21"), а не расписание и список одноклассников — это ссылка (**FOREIGN KEY**) на другую таблицу. Но чтобы **распечатать список студентов вместе с названием их группы**, недостаточно посмотреть только в одну таблицу — нужно "соединить" билеты с журналом групп по этому номеру. Именно это и делает **JOIN**: берёт строки из двух (или больше) таблиц и сшивает их вместе там, где совпадает значение ключа (обычно **PRIMARY KEY = FOREIGN KEY**).

Без **JOIN** пришлось бы делать два отдельных запроса и сшивать результат вручную в коде — **JOIN** делает это одним SQL-запросом, средствами самой базы данных.

Схема для примеров

Все примеры ниже используют упрощённую версию схемы этой платформы:

```
CREATE TABLE courses (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL  
);  
  
CREATE TABLE assignments (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  course_id INTEGER REFERENCES courses(id),  
  title TEXT NOT NULL  
);
```

courses.id		title
1		Веб-разработка
2		Базы данных
assignments.id	course_id	title
1	1	Вёрстка формы
2	1	Адаптивная сетка
3	2	Нормализация БД
4	NULL	Черновик без курса

INNER JOIN — только совпадения (JOIN по умолчанию)

Возвращает только те строки, для которых нашлась пара в обеих таблицах. Слово **INNER** можно не писать — просто **JOIN** означает то же самое.

```
SELECT assignments.title, courses.title AS course_title  
FROM assignments  
INNER JOIN courses ON assignments.course_id = courses.id;
```

Результат — 3 строки (задания 1, 2, 3). Задание "Черновик без курса" **не попадёт** в результат: у него **course_id = NULL**, пары в **courses** для него нет.

title	course_title
Вёрстка формы	Веб-разработка

title	course_title
Адаптивная сетка	Веб-разработка
Нормализация БД	Базы данных

LEFT JOIN — все строки слева, даже без пары

Возвращает **все** строки левой (первой после **FROM**) таблицы. Если пары справа не нашлось, столбцы правой таблицы заполняются **NULL**, но сама строка не пропадает.

```
SELECT assignments.title, courses.title AS course_title
FROM assignments
LEFT JOIN courses ON assignments.course_id = courses.id;
```

title	course_title
Вёрстка формы	Веб-разработка
Адаптивная сетка	Веб-разработка
Нормализация БД	Базы данных
Черновик без курса	NULL

Это ключевое отличие от **INNER JOIN**: "черновик без курса" остался в результате. **LEFT JOIN** часто используют именно для того, чтобы найти строки **без** пары — например, все задания без курса:

```
SELECT assignments.title
FROM assignments
LEFT JOIN courses ON assignments.course_id = courses.id
WHERE courses.id IS NULL; -- пары не нашлось
```

Полное название — **LEFT OUTER JOIN**, слово **OUTER** необязательно и обычно опускается.

RIGHT JOIN и FULL JOIN

- **RIGHT JOIN** — зеркало **LEFT JOIN**: оставляет все строки **правой** таблицы, даже без пары слева.
- **FULL JOIN** (он же **FULL OUTER JOIN**) — оставляет все строки из **обеих** таблиц, независимо от того, нашлась пара или нет.

```
SELECT courses.title, assignments.title
FROM assignments
RIGHT JOIN courses ON assignments.course_id = courses.id; -- все курсы, даже
без единого задания
```

Важно для SQLite: **RIGHT JOIN** и **FULL JOIN** появились только в SQLite версии 3.39 (2022 год) — в старых версиях их нет, и запрос завершится синтаксической ошибкой. На практике вместо **RIGHT JOIN A ON ...** почти всегда можно поменять таблицы местами и использовать привычный **LEFT JOIN**:

```
-- Эквивалент "RIGHT JOIN courses" выше, но через LEFT JOIN — работает в любой
-- версии SQLite:
SELECT courses.title, assignments.title
FROM courses
LEFT JOIN assignments ON assignments.course_id = courses.id;
```

CROSS JOIN — декартово произведение

Соединяет **каждую строку** одной таблицы с **каждой строкой** другой, без всякого условия. Результат — произведение количества строк (3 курса × 4 задания = 12 строк), в реальных задачах используется редко (например, чтобы сгенерировать все возможные комбинации размер/цвет для товара).

```
SELECT courses.title, assignments.title
FROM courses
CROSS JOIN assignments; -- 12 строк: каждый курс со всеми заданиями подряд, без
всякой связи
```

Частая ошибка — случайно получить **CROSS JOIN**, забыв условие **ON** в обычном **JOIN**: SQL это не запрещает, просто молча перемножит все строки, и результат окажется в разы больше ожидаемого.

USING — сокращение для одинаковых имён столбцов

Если столбец связи называется одинаково в обеих таблицах (например, оба называются `course_id`), вместо **ON a.course_id = b.course_id** можно написать короче:

```
SELECT * FROM assignments
JOIN enrollments USING (course_id);
```

Соединение нескольких таблиц подряд

JOIN можно продолжать цепочкой — каждый следующий **JOIN** присоединяется к уже накопленному результату:

```
SELECT students.name, courses.title, submissions.status
FROM submissions
JOIN assignments ON submissions.assignment_id = assignments.id
JOIN courses ON assignments.course_id = courses.id
JOIN students ON submissions.student_id = students.id;
```

Псевдонимы таблиц (алиасы) и самосоединение (SELF JOIN)

Чтобы не писать длинные имена таблиц в каждом условии, им дают короткие псевдонимы через **AS** (**AS** можно опустить):

```
SELECT a.title, c.title AS course_title
FROM assignments AS a
JOIN courses AS c ON a.course_id = c.id;
```

Псевдонимы обязательны, когда таблица соединяется **сама с собой** (self join) — например, чтобы для каждого студента найти его "напарника" из той же таблицы `students`:

```
SELECT s1.name AS student, s2.name AS partner
FROM students AS s1
JOIN students AS s2 ON s1.partner_id = s2.id;
```

Без алиасов SQLite не смог бы понять, о каком именно "экземпляре" таблицы `students` идёт речь в каждой части запроса.

Частые ошибки

- Забуть условие **ON** — запрос превращается в **CROSS JOIN**, результат разрастается в разы, а данные теряют смысл (каждая строка слева склеена с каждой строкой справа).
- Перепутать **INNER JOIN** и **LEFT JOIN** — если нужны в том числе строки "без пары" (например, студенты без единой сдачи), только **LEFT JOIN** их сохранит.

- Обращение к столбцу без указания таблицы, когда он есть в обеих таблицах (например, оба имеют `id`) — SQLite вернёт ошибку "ambiguous column name", нужно явно писать `courses.id` или `assignments.id`.
- Использование `RIGHT JOIN`/`FULL JOIN` без проверки версии SQLite — на старых версиях это синтаксическая ошибка, надёжнее переписать через `LEFT JOIN`, поменяв таблицы местами.